

Architecture Définitive – DAR_Cyber

1. Introduction

L'architecture du projet DAR_Cyber repose sur une approche centralisée de type SOC (Security Operations Center). L'utilisateur (Admin Sys SOC) accède à l'application via une interface web (frontend), qui communique avec un backend FastAPI. Ce backend centralise la logique métier, interagit avec la base de données PostgreSQL et délègue les traitements lourds à Redis et Celery.

2. Description technique globale

Le système est composé de plusieurs couches. L'utilisateur interagit avec le frontend qui envoie des requêtes HTTP vers le backend. Le backend valide les données avec Pydantic, les traite et les stocke dans PostgreSQL. Les traitements complexes comme la corrélation ou la détection sont exécutés de manière asynchrone via Celery.

3. Composants et rôles

Les composants principaux du système sont décrits ci-dessous :

Composant	Rôle
Admin Sys SOC	Utilisateur qui exploite l'application et analyse les alertes
Frontend	Interface utilisateur pour visualiser les données
Backend FastAPI	Logique métier, API REST, validation des données
PostgreSQL	Stockage persistant des données
Redis	Broker de messages pour Celery
Celery	Exécution des tâches asynchrones
Endpoint Agent	Collecte des données réseau
Attacker	Simulation d'attaques pour tests

4. Flux techniques

Les flux techniques permettent de comprendre le fonctionnement global du système :

Étape	Description
1	L'endpoint envoie des événements au backend
2	Le backend valide et stocke les données
3	Une tâche est envoyée à Redis
4	Celery traite la tâche
5	Une alerte est générée
6	Le frontend affiche les résultats

5. Modules backend

Le backend est structuré en modules métier :

Module	Responsabilité
auth	Gestion des utilisateurs et authentification
telemetry	Collecte des données endpoint
assets	Gestion des actifs réseau
alerts	Gestion des alertes
correlation	Corrélation des événements
reports	Exports et reporting
audit	Traçabilité et logs

6. Justification des choix techniques

FastAPI a été choisi pour sa performance et sa simplicité d'utilisation. PostgreSQL est utilisé pour sa robustesse et ses capacités relationnelles. Redis et Celery permettent de gérer les traitements asynchrones afin d'éviter de bloquer l'API.

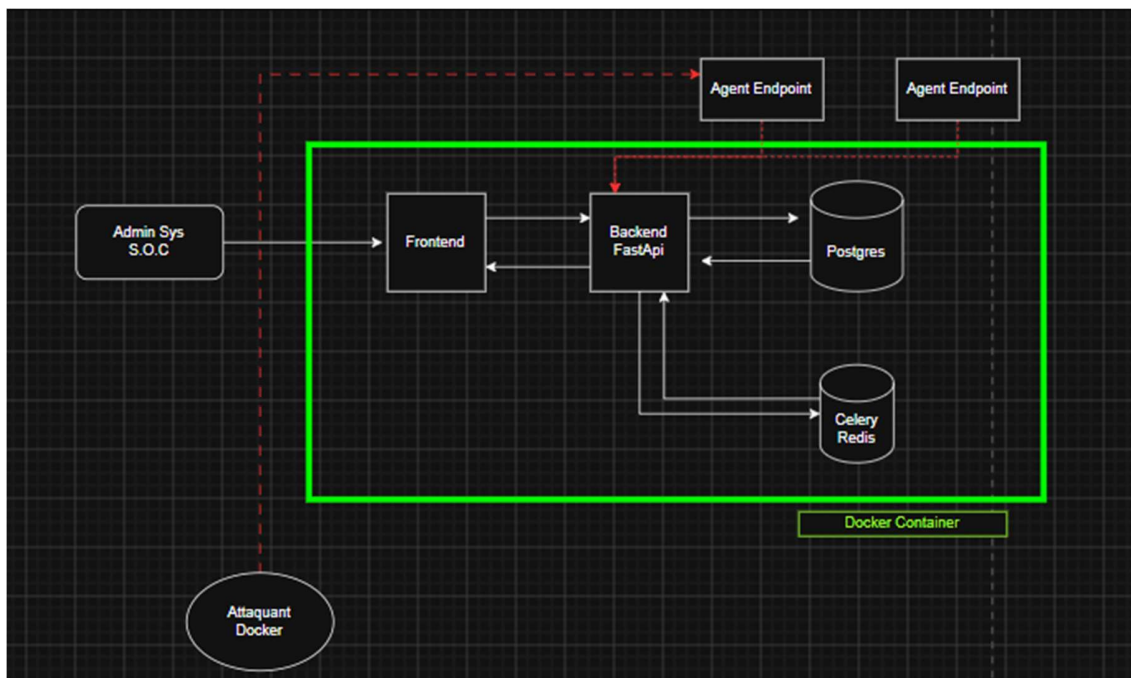
7. Sécurité

La sécurité repose sur plusieurs mécanismes. L'authentification utilisateur est obligatoire. Les rôles permettent de limiter les accès. Les endpoints sont séparés entre utilisateurs et agents. Les données entrantes sont validées pour éviter les injections. Les actions sensibles sont journalisées.

8. Architecture Docker

Le système est déployé via Docker Compose avec trois conteneurs principaux. Le serveur SOC contient le backend, la base de données et les services Redis/Celery. Le réseau Docker isole les composants. Chaque service communique via des noms de service internes.

9. Schema de l'architecture



10. Conclusion

Cette architecture est cohérente, modulaire et adaptée à un projet de cybersécurité. Elle permet de démontrer une chaîne complète de détection et d'analyse.